

보충자료

캡슐화



사전지식

하나의 소스코드를
두 개의 Code구역으로 분류할 수 있다.

Server Code

- 요청을 받아서 동작되는 코드
- Library 또한 Server Code의 대표적인 예시이다.

Client Code

- Server Code를 사용하는 코드

Server Code

```
class GameMachine {  
    public int coin = 0;  
  
    public void insertCoin() {  
        coin += 1;  
    }  
  
    public void playGame() {  
        if (coin == 0) {  
            System.out.println("코인을 넣어주세요");  
            return;  
        }  
        coin -= 1;  
    }  
}
```

Client Code

```
class Main {  
    public static void main(String[] args) {  
        GameMachine gm = new GameMachine();  
        gm.insertCoin();  
        gm.insertCoin();  
    }  
}
```

챕터의 목적 : 캡슐화

캡슐화는 객체지향 패러다임에서 지켜야할 중요한 원칙이다.

Server Code를 만들 때, Client Code가 몰라도 되는 내용은 모르게 만들고 꼭 필요한 내용만 제공하여 안전한 프로그래밍이 되도록 하는 것이다.

이번 챕터에서는 캡슐화의 대표적인 세 가지 사례에 대해 알아본다.

1. Client Code에게 불필요한 정보 은닉하기
2. Getter와 Setter 사용하기
3. 구체적인 내용 숨기고, 접근방법 제공하기

1. Client Code에게 불필요한 정보 은닉하기

쉬운 이해를 위한 상황 가정

B 부서에서 Server Code를 개발했다고 가정하자.

A 부서에서는 Server Code를 사용하는 Client Code를 작성해야 한다.

Server Code
B 부서 담당

```
class GameMachine {  
    public int coin = 0;  
  
    public void insertCoin() {  
        coin += 1;  
    }  
  
    public void playGame() {  
        if (coin == 0) {  
            System.out.println("코인을 넣어주세요");  
            return;  
        }  
        coin -= 1;  
    }  
}
```

Client Code
A 부서 담당

```
class Main {  
    public static void main(String[] args) {  
        GameMachine gm = new GameMachine();  
        gm.insertCoin();  
        gm.insertCoin();  
    }  
}
```

이슈 발생!

Server Code를 개발한 B 부서에서
예상하지 못한 방법으로 사용되고 있다.

이 버그는 누구의 책임일까?

1. A부서 : Client가 잘못 사용한 것이 문제이다.
2. B부서 : Server가 자유도를 너무 높게 만든것이 문제이다.

```
class GameMachine {
    public int coin = 0;

    public void insertCoin() {
        coin += 1;
    }

    public void playGame() {
        if (coin == 0) {
            System.out.println("코인을 넣어주세요");
            return;
        }
        coin -= 1;
    }
}

class Main {
    public static void main(String[] args) {
        GameMachine gm = new GameMachine();
        gm.coin = 999; //으잉?
        gm.insertCoin();
        gm.insertCoin();
    }
}
```

B 부서가 잘못 만든 것이다.

public은 클래스 외부에서 호출해도 된다는 의미
coin은 외부에서 건드리면 안되는 데이터이다.

따라서 B가 잘못된 권한을 많이 준 것이 문제다.

예시)

은행의 돈을 외부 사람이 건드리지 못하도록 해야한다.

돈을 바깥에 두면 (Public 설정)

가져간 사람보다 은행이 돈 관리를 잘못된 것이다.

```
class GameMachine {  
    public int coin = 0;  
  
    public void insertCoin() {  
        coin += 1;  
    }  
  
    public void playGame() {  
        if (coin == 0) {  
            System.out.println("코인을 넣어주세요");  
            return;  
        }  
        coin -= 1;  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        GameMachine gm = new GameMachine();  
        gm.coin = 999; //으잉?  
        gm.insertCoin();  
        gm.insertCoin();  
    }  
}
```

private 접근제어자 사용하기 1

private을 활용하여,
공개하지 않을 데이터를 지정한다.

클래스 개발시 주의사항

내부에서 사용되는 데이터를
public 으로 두는 행동은
버그를 유발시키는 행동이다.

```
class GameMachine {  
    private int coin = 0;  
  
    public void insertCoin() {  
        coin += 1;  
    }  
  
    public void playGame() {  
        if (coin == 0) {  
            System.out.println("코인을 넣어주세요");  
            return;  
        }  
        coin -= 1;  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        GameMachine gm = new GameMachine();  
        gm.coin = 999; //으잉?  
        gm.insertCoin();  
        gm.insertCoin();  
    }  
}
```


private 접근제어자 사용하기 2

private을 활용하여,
내부에서만 사용되는 멤버함수도 만들 수 있다.

클래스 개발시 주의사항

Client가 사용하라고 만든 의도가 아닌 것은
모두 private 멤버함수로 만들어야한다.

Client Code 개발자가 이거 어디다 쓰지? 혼동이 된다.
잘못된 사용으로 버그도 유발될 수 있다.

```
class GameMachine {  
    private int coin = 0;  
  
    public void insertCoin() {  
        coin += 1;  
    }  
  
    public void playGame() {  
        if (!hasCoin()) {  
            System.out.println("코인을 넣어주세요");  
            return;  
        }  
        coin -= 1;  
    }  
  
    private boolean hasCoin() {  
        return coin > 0;  
    }  
}
```

2. Getter / Setter

데이터의 접근 권한 관리

현실에서 사용되는 데이터는
세 가지 접근 권한으로 관리되곤 한다.

1. 읽기 전용 데이터 (스포츠 경기 점수)
2. 쓰기 전용 데이터 (기부금 넣기)
3. 읽고 쓰기 모두 가능한 데이터 (은행계좌)

프로그래밍에서도
getter와 setter를 이용하여 데이터 접근 권한 관리가 가능하다.

데이터 읽기 전용

데이터 값을 읽는 멤버함수를 Getter라고 한다.
Getter만 제공하는 경우,
Client Code에게 name과 age 데이터를
읽기만 허용하는 것이다.

```
class Person {  
    private String name;  
    private int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
}
```

데이터 쓰기 전용

데이터 값을 변경하는 멤버함수를 Setter라고 한다.
Getter없이 Setter만 제공하는 경우
name과 age 데이터는
쓰기 전용 데이터가 된다.

```
class Person {  
    private String name;  
    private int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public void setAge(int age) {  
        this.age = age;  
    }  
}
```

Setter로 방어적 프로그래밍 가능

Client가 잘못된 값을 넣었을 때,
빠르게 버그를 확인할 수 있는
방어코드를 삽입할 수 있다.

```
class Person {  
    public String name;  
    public int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Person g = new Person("HI", 20);  
        g.name = "";  
        g.age = -99;  
    }  
}
```

Server Code가 의도하지 않은 방향으로
Client Code가 잘못 사용할 수 있음

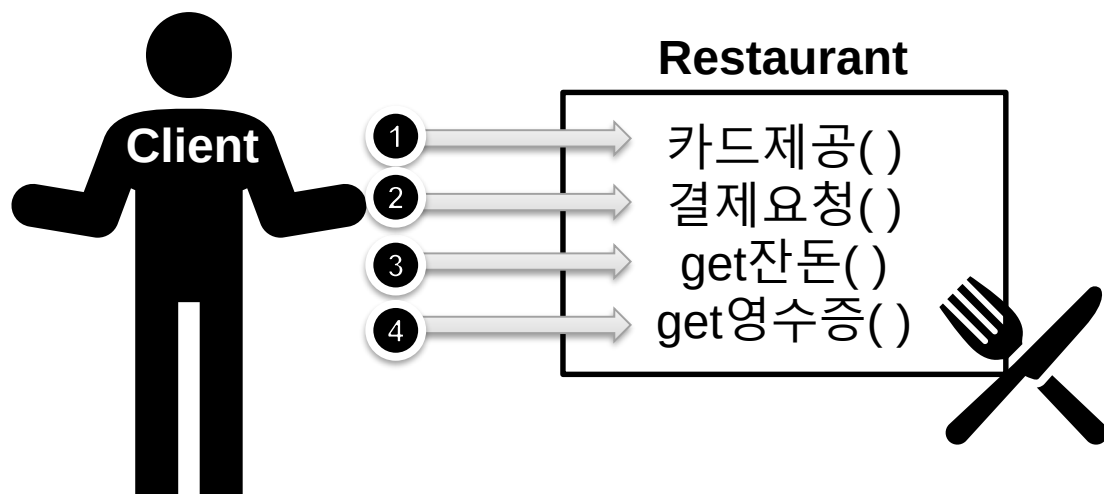
```
class Person {  
    private String name;  
    private int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    void setName(String name) {  
        if (name.isEmpty()) {  
            System.out.println("이름은 한글자 이상이 되어야합니다.");  
            return;  
        }  
        this.name = name;  
    }  
  
    void setAge(int age) {  
        if (age < 0) {  
            System.out.println("나이는 음수가 될 수 없습니다.");  
            return;  
        }  
        this.age = age;  
    }  
}
```

Client Code가 ServerCode를 잘못 사용하는 경우
버그 원인을 빠르게 확인할 수 있다.

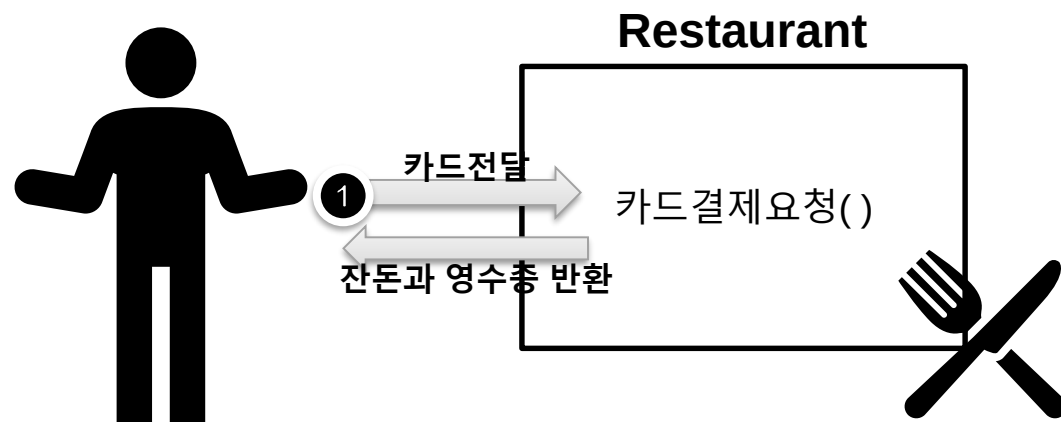
3. 구체적인 동작방법 숨기고, 접근방법 제공하기

구체적인 동작 방법을 숨긴다.

Client에게 구체적인 동작 방법을 숨기고,
Client를 위한 접근 방법을 제공한다.



캡슐화가 **잘 안된** 예시
Client가 1 ~ 4 번 순서대로
호출해야 한다는 것을 알고 있어야만 한다.



캡슐화가 **잘 되어**있는 예시
Server Code는 구체적인 방법을 숨겼으며,
Client를 위한 접근 방법을 제공한다.

캡슐화가 덜 되어있는 코드 예시

Client Code를 만들 때,
Server Code를 구체적으로 알고 있어야
버그가 발생되지 않는다.

Server Code

```
class Restaurant {  
    public void 카드제공(int card) { }  
    public void 결제요청() { }  
    public int get잔돈() { return 0; }  
    public int get영수증() { return 0; }  
}
```

Client Code

```
class Customer {  
    private int myCard = 0;  
    private int myCash = 0;  
    private int myBill = 0;  
  
    public void haveDinner(Restaurant restaurant) {  
        restaurant.카드제공(myCard);  
        restaurant.결제요청();  
        myCash += restaurant.get잔돈();  
        myBill = restaurant.get영수증();  
    }  
}
```

Client가
결제를 위한 순서를
알고있어야한다.

캡슐화가 잘 되어있는 코드 예시

Server는 구체적인 내용을 숨기고 접근방법을 제공한다.

Client는 구체적인 내용을 몰라도 쉽고 안전한 접근이 가능하다.

```
class Customer {  
    private int myCard = 0;  
    private int myCash = 0;  
    private int myBill = 0;  
  
    public void haveDinner(Restaurant restaurant) {  
        Response reponse = restaurant.카드음식결제(myCard);  
        myCash = reponse.get잔돈();  
        myBill = reponse.get영수증();  
    }  
}
```

Client Code

```
class Response {  
    private int 잔돈;  
    private int 영수증;  
  
    public Response(int 잔돈, int 영수증) {  
        this.잔돈 = 잔돈;  
        this.영수증 = 영수증;  
    }  
  
    public int get잔돈() {  
        return 잔돈;  
    }  
  
    public int get영수증() {  
        return 영수증;  
    }  
}
```

Server Code

```
class Restaurant {  
    public Response 카드로음식결제(int customerCard) {  
        카드제공(customerCard);  
        결제요청();  
        int charge = get잔돈();  
        int bill = get영수증();  
        return new Response(charge, bill);  
    }  
  
    private void 카드제공(int card) { }  
    private void 결제요청() { }  
    private int get잔돈() { return 0; }  
    private int get영수증() { return 0; }  
}
```

[정리] 캡슐화

Server Code 기준으로 불필요한 정보를 숨기고,
접근할 수 있는 함수를 제공하는 것을 캡슐화 라고 한다.

Client에게 숨겨야 할 불필요한 정보는 다음과 같다.

- 보여줄 필요없는 데이터
- Server 내부적으로만 쓰이는 함수
- 구체적인 동작 방식

캡슐화를 통해 다음과 같은 장점을 얻을 수 있다.

- 안전성 확보 : getter / setter를 이용하여 데이터별 접근권한을 부여 가능
- Client Code에서 Server Code 사용이 쉬워짐
- 유지보수성 좋아짐